

DNA Sequence Database Extension

PostgreSQL Extension for DNA & k-mer Analysis

El Kazbani Othman

Tektas Rahman

Touimer Amin

Ziane Yasser

Université libre de Bruxelles (ULB)
INFO-H417 — Database System Architecture

December, 2025

- 1 Introduction
- 2 Repository Structure
- 3 Data Types & Representation
- 4 Functions & Operators
- 5 K-mer Counting Support
- 6 SP-GiST Trie Index
- 7 Testing & Conclusion

- PostgreSQL extension providing 3 custom bioinformatics types:
 - **dna** — variable-length DNA (A,C,G,T)
 - **kmer** — fixed-length 1-32, A/C/G/T only
 - **qkmer** — same length, with IUPAC ambiguity codes
- Implemented fully in C as a `MODULE_big` extension.
- Features:
 - In/out functions, `length`, `generate_kmers`
 - Operators: `=`, `^@`, `@>`
 - Hash and B-tree opclasses for `kmer`
 - SP-GiST trie index for equality/prefix/pattern search

- K-mers are central in:
 - assembly, variant detection, read mapping
- PostgreSQL benefits:
 - SQL-native k-mer generation and aggregation
 - Efficient prefix queries and IUPAC pattern queries
 - Integration with indexes and query planning
- Our goal: **high-performance k-mer operations directly inside PostgreSQL.**

- `dna_sequence/` — PostgreSQL extension source
 - `dna_sequence.c` — core types & functions
 - `dna_sequence.h` — structs, bitmasks, helpers
 - `spgist.c` — SP-GiST trie implementation
 - `dna_sequence-1.0.sql` — SQL bindings & opclasses
 - `dna_sequence.control` — extension metadata
 - `Makefile` — PGXS build & Docker integration
- `tests/` — SQL validation scripts
 - `test_dna.sql`, `test_kmer.sql`, `test_qkmer.sql`
 - `test_spgist.sql` — index correctness tests
- **Repository root**
 - `README.md` — build and usage instructions
 - `presentation.pdf` — deliverable

- Struct:

```
typedef struct {  
    int32    vl_len_;  
    uint8_t  code[];  
} Dna;
```

- Parsing:

- `dna_parse()` accepts A,C,G,T only.
- length range: 1-255.

- Storage:

- bases stored as 4-bit masks (same encoding as `kmer`).
- mapping done by `char_to_mask(c, true)`.

- `length(dna)` uses `VARSIZE_ANY_EXHDR()`.

kmer

- Struct:

```
int32    vl_len_;  
uint8_t  code[];
```

- Length: 1-32.
- Each base stored as a 4-bit mask:

```
BASE_A = 0x01  
BASE_C = 0x02  
BASE_G = 0x04  
BASE_T = 0x08
```

- `kmer_parse()` uses strict mapping (A,C,G,T only).

qkmer

- Same structure, same length range.
- IUPAC masks (subsets of A,C,G,T):

$$Y = C|T$$
$$R = A|G$$
$$V = A|C|G$$
$$N = A|C|G|T$$

- `qkmer_parse()` accepts ambiguous codes and stores them as bitmasks.

- `char_to_mask(c, strict)`
 - strict mode → only A,C,G,T (dna, kmer)
 - non-strict → IUPAC expanded to OR-combinations (qkmer)
- `mask_to_char(mask)`
 - used by out-functions to rebuild dna, kmer, qkmer
- Pattern matching becomes simple bitwise logic:
 $(q_mask \& k_mask) \neq 0$

- In/out:
 - dna_in/out, kmer_in/out, qkmer_in/out

- Length for all types uses:

`VARSIZE_ANY_EXHDR(value)`

- SQL binding example:

```
CREATE FUNCTION length(kmer)
RETURNS int
AS 'MODULE_PATHNAME', 'kmer_length'
LANGUAGE C IMMUTABLE STRICT;
```

- Implemented as SRF with FuncCallContext.
- Context stores:
code, dna_length, k, num_kmers
- First call:
 - detoast DNA, compute num_kmers
- Each iteration:
 - copy the mask-encoded slice into a fresh `kmer`
 - return as SETOF `kmer`

- **equals(kmer,kmer)** compares lengths + memcmp on code[].
- **starts_with(kmer,kmer)** prefix length check + memcmp on prefix.
- **contains(qkmer,kmer)** bitwise match per position:
 $(q_mask \ \& \ k_mask) \ != \ 0$

```
CREATE OPERATOR = (  
    LEFTARG = kmer, RIGHTARG = kmer,  
    PROCEDURE = equals  
);
```

```
CREATE OPERATOR ^@ (  
    LEFTARG = kmer, RIGHTARG = kmer,  
    PROCEDURE = starts_with  
);
```

```
CREATE OPERATOR @> (  
    LEFTARG = qkmer, RIGHTARG = kmer,  
    PROCEDURE = contains  
);
```

These operators also power B-tree, hash, and SP-GiST opclasses.

- `kmer_hash()` uses:

```
hash_any(kmer->code, kmer_length)
```

- SQL wrapper:

```
CREATE FUNCTION hash(kmer)
RETURNS int
AS 'MODULE_PATHNAME', 'kmer_hash'
LANGUAGE C IMMUTABLE STRICT;
```

- Opclass:

```
CREATE OPERATOR CLASS kmer_hash_ops
DEFAULT FOR TYPE kmer USING hash AS
  OPERATOR 1 =,
  FUNCTION 1 hash(kmer);
```

Counting Examples

```
SELECT k.kmer, count(*)  
FROM generate_kmers('ACGTACGT', 5) AS k(kmer)  
GROUP BY k.kmer  
ORDER BY count(*) DESC;
```

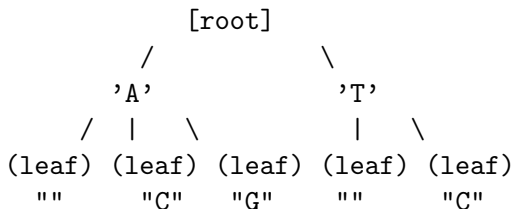
```
WITH kmers AS (  
    SELECT k.kmer, count(*) AS c  
    FROM generate_kmers(dna, 5) AS k(kmer)  
    GROUP BY k.kmer  
)  
SELECT sum(c), count(*),  
       count(*) FILTER (WHERE c = 1)  
FROM kmers;
```

- K-mers are prefix-based → natural trie structure.
- SP-GiST allows custom tries with:
 - dynamic branching
 - compact shared prefixes
- Our index supports:
 - equality: `kmer = kmer`
 - prefix: `kmer ^@ kmer`
 - IUPAC pattern: `qkmer @> kmer`

- Each inner node stores:
 - an optional **prefix** (common to all children)
 - an array of **child labels** (one byte or -1 = end-of-kmer)
- Each leaf stores the remaining suffix of the k-mer.
- Path from root to leaf = full k-mer.

Trie Example on Small Dataset

- Demo table: { 'A', 'AC', 'AG', 'T', 'TC' }
- Our SP-GiST implementation stores:
 - **inner node prefix**: shared substring
 - **node labels**: next base or -1 (end-of-kmer)
 - **leaf payload: suffix only** (never full k-mer)



- Strategy numbers (in C):

```
KMER_EQUAL_STRATEGY      = 1  -- kmer = kmer  
KMER_PREFIX_STRATEGY     = 2  -- kmer ^@ kmer  
QKMER_CONTAINS_STRATEGY  = 3  -- qkmer @> kmer
```

- **spg_kmer_config**

- prefixType = leafType = kmer
- labelType = int2 (one byte or sentinel)

- **formKmerDatum / commonPrefix**

- helpers to build prefixes/suffixes as varlena kmer
- used by choose, picksplit, and inner_consistent

- **spg_kmer_choose**

- detoasts input k-mer and current node prefix
- computes common prefix length
- either:
 - requests a **split** of the existing prefix, or
 - routes to an existing child (match node), or
 - creates a new child (add node)

- **spg_kmer_picksplit**

- computes global common prefix across all tuples
- sets it as node prefix (capped by SPGIST_MAX_PREFIX_LENGTH)
- groups tuples by next byte (nodeLabels)
- stores remaining suffix as the leaf payload

- **spg_kmer_inner_consistent**

- reconstructs candidate prefix for each child
- applies all query keys:
 - `=` → prefix must match query and not exceed it
 - `^@` → prefix must be consistent with query prefix
 - `@>` → each base compatible with qkmer mask
- prunes children that can never match

- **spg_kmer_leaf_consistent**

- reconstructs full k-mer = prefix + leaf suffix
- performs final check:
 - full equality
 - full prefix test
 - full qkmer pattern test
- sets `recheck = false` (index is exact)

Using the Index

```
CREATE INDEX idx_kmers_spgist
  ON kmers USING spgist(kmer);

-- equality
SELECT * FROM kmers WHERE kmer = 'ACGTA';

-- prefix
SELECT * FROM kmers WHERE kmer ^@ 'ACG';

-- IUPAC pattern
SELECT * FROM kmers
WHERE 'ANGTA'::qkmer @> kmer;
```

- **Environment**

- Docker container: `info-h417-dna-extension`.

- **Main commands**

- `make docker-reinstall`
→ clean, build, install, recreate extension
- `make run`
→ run all tests (DNA, KMER, QKMER, SP-GiST)
- `make test-dna, test-kmer, test-qkmer, test-spgist`

- **Helper commands**

- `make docker-sync-scriptpy` — sync Python scripts
- `make psql` — open `psql` in the container
- `make docker-clean` — remove compiled artifacts

- **Synthetic tests** (SQL only)

- Small hand-written sequences
- Check: parsing, length, generate_kmers
- K-mer counting with GROUP BY
- SP-GiST on toy table `kmer_test`

- **Real sequencing data** (NCBI SRA)

- Download FASTQ with SRA Toolkit
- Python scripts:
 - `extract_dna.py` → `dna_36.txt` (clean reads, no N)
 - `generate_qkmer.py` → `qkmeroutput.txt` (qkmers with IUPAC)
- COPY into PostgreSQL:
 - `test_dna(seq)` from `dna_36.txt`
 - `qkmer_targets_dataset(val)` from `qkmeroutput.txt`
- Generate kmers, build SP-GiST index, run:
 - `equality(=)`
 - `prefix(^@)`
 - pattern queries (`@>`) with `EXPLAIN ANALYZE`

- Three custom types:
 - dna — bitmasks, strict A/C/G/T
 - kmer — bitmasks, strict, 1–32 bases
 - qkmer — bitmasks + IUPAC ambiguity
- Functions & operators:
 - length, generate_kmers
 - equals, starts_with, contains
- Indexing:
 - hash + btree for counting and sorting
 - SP-GiST trie for equality, prefix, and pattern queries

- `generate_kmers` creates one varlena per k-mer, which is fine for small inputs but costly for millions of k-mers.
- No parallelism: all parsing, k-mer generation, and indexing run in a single thread.

Future improvements:

- parallel k-mer generation

Questions?